

HelixQA Runner Readiness — can the matrix banks DRIVE the real app flow?

Question. Can the HelixQA matrix banks drive the real Lava app flow (onboard → search 1080p / mp3 → open details → obtain download link) on a containerized emulator via **adb + vision OCR** — i.e. **without** `createAndroidComposeRule`, so immune to the upstream LVA-008 Compose-rule teardown defect — as an alternative/complementary evidence vehicle to the Compose Challenge?

One-line verdict. VIABLE-WITH-GAPS. The adb + vision-driven, Compose-rule-free *capability* exists and is unit-tested in the pinned HelixQA submodule (pkg/visionnav + pkg/navigator + pkg/autonomous), but the **integration glue that drives a MATRIX BANK through that vision loop is not wired in the pinned submodule** — so neither mode the Lava script offers (`--matrix` nor `--autonomous`) actually executes the pinned 1080p/mp3 journey end-to-end today.

This is an evidence-based finding from reading the pinned submodule source. Where a fact could not be proven from code it is marked UNCONFIRMED: per §11.4.6 (no-guessing).

1. Does HelixQA actively drive UI input, or only launch + passively OCR?

Both capabilities exist in the codebase, but they live in DIFFERENT code paths, and the matrix-bank path that the Lava script invokes does NOT reach the active-driving one.

1a. The active-driving mechanism EXISTS (and is Compose-rule-free)

The vision-driven loop genuinely drives input — tap / type / key / back / home / swipe — under LLM control, using only adb + screenshots. No `createAndroidComposeRule`, no instrumentation, no

UIAutomator-dump dependency; the taps are **vision-located coordinate taps** the LLM produces from looking at each screenshot.

- pkg/visionnav/session.go:118 — Session.Run loop, per step: screenshot → build Observation{LastImageBytes} → Provider.Decide(obs) → **Actor.Dispatch(action)** → Explorer.CaptureFinding → check ScreenGoals against the captured OCR snapshot.
- pkg/visionnav/llm_provider.go:110 — LLMPProvider.Decide sends the screenshot **bytes + a goal-aware prompt** to a vision-capable LLM (VisionDecider.Vision) and parses an ACTION / RATIONALE / EXPECT reply into a validated Decision (a reply with no action/rationale is rejected as a bluff — :217, :221).
- pkg/visionnav/adb_actor.go:87 — ADBActor.Dispatch maps the action grammar (tap <x> <y>, text <string>, key <KEYCODE>, back, home, launch, shell) onto a DeviceExecutor. **An unknown verb is a hard error, never a silent no-op (:140)** — the anti-bluff posture that prevents "thought it acted but didn't".
- pkg/navigator/executor.go — ADBExecutor implements the real device primitives: Tap via cmd input tap with legacy input tap fallback (:113-127), Type via adb shell input text (:135), KeyPress/Back/Home via input keyevent (:164-282), Swipe via input swipe (:235), Screenshot via adb -s <serial> exec-out screencap -p with a shell screencap fallback (:351-359). All over adb — **no Compose rule.**
- Anti-bluff verdict gate: session.go:180 — a run PASSes only when **both** (a) a captured Evidence.OCRSnapshot matches a registered ScreenGoal **and** (b) the screen demonstrably changed between steps (zero-screen-delta ⇒ auto-FAIL, §11.4.52). This is exactly the "the action did something the user can see" guarantee.

Conclusion: the active-driving + vision-OCR + anti-bluff mechanism is real and Compose-rule-free → architecturally LVA-008-immune.

1b. The matrix-bank run path only LAUNCHES + passively checks — it does NOT drive

The Lava --matrix path runs helixqa run --banks <matrix> --platform android (scripts/run-helixqa-provider-qa.sh:95). That command goes through orchestrator.Run → runPlatform("android"):

- cmd/helixqa/main.go:187,210 — run constructs orchestrator.New(...) and calls orch.Run(ctx). The orchestrator package is the only run engine here.
- pkg/orchestrator/orchestrator.go:300-356 — per definition it calls a definitionChallenge.Execute. For android (anything but desktop) that returns an **honest SKIP** (definition_challenge.go:390-402, skippedResult reason: "bank

*case has no executable action for the "android" platform (needs an Android/UI/web topology backend)", :498-501). It **never constructs a visionnav.Session and never drives a tap/type.***

- promoteSkippedToPassed (orchestrator.go:399, logic definition_challenge.go:146-194) CANNOT rescue these cases: the matrix cases carry adb_shell: launch steps + tap steps, and caseHasUnrunAssertingStep (:204-226) returns true for any adb_shell / tap / text action → promotion is **blocked** → the cases stay **SKIPPED**.
- The validator that does run (orchestrator.go:360-384, pkg/detector/android.go) is **passive crash/ANR detection** (pidof / logcat / screencap), not input driving.
- The bank's dispatches_to: "monkey -p ... 1" and required_evidence tokens are **NOT consumed by orchestrator.Run at all** — that metadata is owned by pkg/testbank.Dispatcher (dispatch.go), which orchestrator.go never imports or calls.

Net for --matrix: every matrix case reports an honest SKIPPED; no onboarding, no search, no details, no download is exercised. (A run with 0 executed challenges is honestly reported as not-success — orchestrator.go:222.)

1c. The bank's "Executor contract" comment is ASPIRATIONAL

The matrix YAML header (lava-rutor-matrix-journey.yaml:29-37) claims:

```
Executor contract (pkg/orchestrator/
definition_challenge.go): platforms:[android] routes each
case to executeAndroidVisionSteps, which builds a
visionnav.Target{LaunchAction, ScreenGoals} ...
deriveLaunchAction ... deriveScreenGoals ...
```

None of executeAndroidVisionSteps, deriveLaunchAction, or deriveScreenGoals exist anywhere in the pinned submodule (verified by exact-symbol grep — zero matches). visionnav.Session / visionnav.NewSession is **never constructed in any non-test production file** (verified). The bank was authored against a planned wiring that the pinned submodule does not yet contain.

1d. The --autonomous mode DOES drive input — but is NOT matrix-bank-driven

helixqa autonomous (scripts/run-helixqa-provider-qa.sh:92) uses a different engine that genuinely drives the device:

- cmd/helixqa/main.go:704 → autonomous.NewSessionPipeline, with executors from pkg/autonomous/real_executor.go:83 →

navigator.NewADBExecutor(serial, ...) (the same tap/type/screencap primitives as 1a). It records video (FFmpegPath / HELIX_FFMPEG_PATH, cfg at main.go:670) and uses LLM vision providers.

- **BUT it is doc-driven, not bank-driven:** cfg.IssuesDir = <project>/docs/issues and cfg.BanksDir = <project>/challenges/helixqa-banks (main.go:657-662). The Lava matrix banks live at lava-api-go/qa/banks/*.yaml, which autonomous mode does **not** read, and <project>/challenges/helixqa-banks does not exist in Lava. So autonomous mode will **not** run the pinned 1080p/mp3 journey; it would free-explore from docs.

1e. The only binary that consumes dispatches_to + required_evidence is a script-runner, not a UI driver

cmd/helixqa-bank-session/main.go:147 wires testbank.Dispatcher. But it (a) is **not** the binary the Lava script builds (run-helixqa-provider-qa.sh:73 builds ./cmd/helixqa), and (b) interprets dispatches_to as a **repo-relative on-device test_*.sh it pushes + runs via adb** (main.go:134-135,209-233), then enforces a content-asserting evidence ledger. That is script-execution + evidence-gating — **still not active onboarding/search/tap UI driving**, and the matrix bank's dispatches_to: "monkey -p ... 1" is not a script path anyway.

2. Exact command to run a matrix bank against a booted emulator

The Lava glue (scripts/run-helixqa-provider-qa.sh) targets adb by serial via --device/\$SERIAL (default 127.0.0.1:6555 Genymotion, or \$LAVA_REAL_DEVICE_SERIALS). For a booted emulator at 127.0.0.1:<port>:

```
# Via the Lava glue (preferred – does the adb-connect + device
gate + build):
scripts/run-helixqa-provider-qa.sh --provider rutor --matrix --
serial 127.0.0.1:<port>
```

```
# Equivalent raw helixqa invocation (what the glue runs
internally):
```

```
helixqa run \
  --banks lava-api-go/qa/banks/lava-rutor-matrix-journey.yaml \
  --platform android \
  --device 127.0.0.1:<port> \
  --package digital.vasic.lava.client.dev \
  --output .lava-ci-evidence/helixqa/rutor/<run-id>
```

--all runs every provider bank; --autonomous switches to the doc-driven engine (see §1d caveat — does not consume the matrix bank).

Caveat (consequence of §1b): today this command returns **SKIPPED rows** for the matrix cases — it does not drive the flow. The command is correct; the runner wiring behind it is the gap.

3. Does it record video + perform vision OCR analysis?

- **Video recording:** Yes, the capability exists. `helixqa run` accepts `--record` (default **true**, `cmd/helixqa/main.go:131`) and the codebase has real recorders (`pkg/session/recorder.go`, `pkg/capture/linux_capture.go`, `ffmpeg` via `HELIX_FFMPEG_PATH`). In `--autonomous` mode video is wired through `cfg.FFMpegPath` (`main.go:670`). **However**, in the `--matrix` run path the android cases SKIP without the app being actively driven, so any recorded video would capture an undriven screen — not useful journey evidence. **UNCONFIRMED:** whether `--record=true` actually starts a screenrecord in the android run path when every case skips (not traced to a start call in `runPlatform`).
 - **Vision OCR analysis:** Yes, the capability exists. The vision loop matches goals against a real Tesseract OCR snapshot (`pkg/visionnav/explorer.go:135-141` `DefaultExplorer` → `tesseract.OCR`, evidence stored via `FileSink`, `evidence.go`). The §11.4.52 OCR-goal + screen-delta gate is in `session.go:174-189`. Autonomous mode additionally uses LLM vision over screenshots directly. **But** — same gap — this OCR analysis only happens inside `visionnav.Session` / the autonomous pipeline, neither of which the `--matrix` run path constructs. The run path's only screen analysis is passive crash/ANR detection (`pkg/detector/android.go`).
-

4. VIABILITY VERDICT

VIALE-WITH-GAPS as the matrix UI-evidence vehicle.

Why viable (the hard part is already built): an adb-only, vision-driven, Compose-rule-free active-driving loop with an anti-bluff goal+screen-delta verdict and Tesseract OCR exists and is unit-tested (`pkg/visionnav/{session, llm_provider, adb_actor, explorer, evidence}.go` + `pkg/navigator/executor.go` + `pkg/autonomous/real_executor.go`). Because it is pure adb + screenshots + vision LLM, it is **structurally immune to the LVA-008** `createAndroidComposeRule teardown`

defect — it never instantiates a Compose rule. The `--autonomous` engine already drives input + records video + uses vision on a device addressed by adb serial.

The specific gaps (what blocks it being the matrix vehicle today):

- 1. No matrix-bank → vision-loop wiring (primary gap).**
orchestrator.Run handles android bank cases by honest-SKIP; it does not build a visionnav.Target from the bank and does not run visionnav.Session. The functions the bank comment references (executeAndroidVisionSteps, deriveLaunchAction, deriveScreenGoals) **do not exist** in the pinned submodule. **Fix:** add an android execution path in the orchestrator that, for platforms:[android] cases, derives LaunchAction from the first adb_shell: step and ScreenGoals from required_evidence + expected_result, then runs visionnav.Session with visionnav.NewADBActor(navigator.NewADBExecutor(serial,...))
 - visionnav.NewLLMProvider(<vision backend>, goals, grammar-hint) + visionnav.NewDefaultExplorer(tesseract, FileSink). This is HelixQA-side (own-org submodule) work, contributable upstream per the Decoupled Reusable Architecture rule.
- 2. Vision backend must be supplied.** The loop needs a VisionDecider (SupportsVision()==true). The Lava script offers the claude CLI bridge (run-helixqa-provider-qa.sh:79-81); UNCONFIRMED: whether the bridged CLI provider advertises SupportsVision()==true end-to-end in run mode (it is wired for autonomous). Without a vision backend the loop cannot decide coordinate taps.
- 3. --autonomous points at the wrong banks/docs for Lava.** It reads <project>/docs/issues + <project>/challenges/helixqa-banks, not lava-api-go/qa/banks/. To use autonomous mode as the matrix vehicle, either place the matrix journeys where autonomous looks, or (better) close gap #1 so the deterministic run --banks path drives them.
- 4. Credentials / anonymous path.** The login case loads RUTOR_USERNAME / RUTOR_PASSWORD from env (lava-rutor-matrix-journey.yaml:96); the bank accepts the anonymous "continue" path too, so a no-credential run can still satisfy the search+download cases — but the vision loop must be able to find and tap the anonymous-continue control.
- 5. Host/topology gate unchanged.** §6.AH requires the emulator to run in a container/VM via the Containers submodule; the Lava script's device gate is an honest

BLOCKED (exit 2) when no adb device is in device state (run-helixqa-provider-qa.sh:57-68). This is orthogonal to the wiring gap but is the runtime precondition.

Bottom line: HelixQA is the right *complementary* vehicle — an adb+vision, Compose-rule-free path that side-steps LVA-008 — and the engine parts exist, but the matrix banks cannot drive the flow until gap #1 (the orchestrator's android-vision wiring) is implemented in the pinned submodule. Until then the --matrix run reports honest SKIPS, and --autonomous does not consume the matrix banks.